

Мета: ознайомитись з використанням AWS DynamoDB (serverless database)

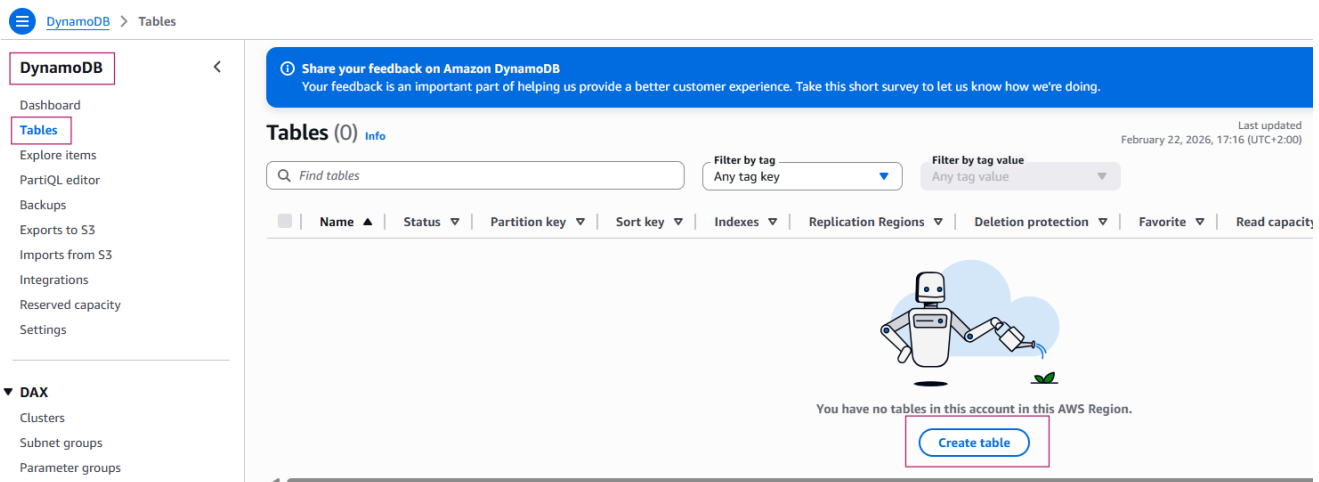
Завдання:

- Спроекувати структуру даних (таблицю)
- Вивчити способи роботи з даними засобами DynamoDB
- Виконати завдання в GUI, засобами AWS CLI та Python

1. Для даних, отриманих в роботі No2 спроектувати таблицю DynamoDB (засобами NOSQL Workbench, AWS CLI та AWS Management Console)

AWS Management Console:

Для початку перейду до DynamoDB =>Tables=>Create table



Створюю нову таблицю: задаю назву таблиці sonkkstable, первинний ключ table типу String, а інші параметри залишаю за замовчуванням

Share your feedback on Amazon DynamoDB

Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.

Create table

Table details [Info](#)

DynamoDB is a schemaless database that requires only a table name and a primary key when you create the table.

Table name

This will be used to identify your table.

sonkkstable

Between 3 and 255 characters, containing only letters, numbers, underscores (_), hyphens (-), and periods (.).

Partition key

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability.

table

String

1 to 255 characters and case sensitive.

Sort key - optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter the sort key name

String

1 to 255 characters and case sensitive.

Перевіряю створення таблиці sonkkstable за допомогою команди `aws dynamodb list-tables`

```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb list-tables
{
  "TableNames": [
    "sonkkstable"
  ]
}
```

AWS CLI

Надаю користувачу політику `AmazonDynamoDBFullAccess` для роботи з сервісом DynamoDB

Permissions policies (4)

Permissions are defined by policies attached to the user directly or through groups.

Filter by Type			
All types			
☐	Policy name ↗	Type	Attached via ↗
☐	 AmazonDynamoDBFullAccess	AWS managed	Directly
☐	 AmazonEC2FullAccess	AWS managed	Directly
☐	 AmazonS3FullAccess	AWS managed	Directly
☐	 AmazonS3ReadOnlyAccess	AWS managed	Directly

Створюю таблицю sonkkstable2 через AWS CLI командою `create-table` із простим первинним ключем `id` типу `String`

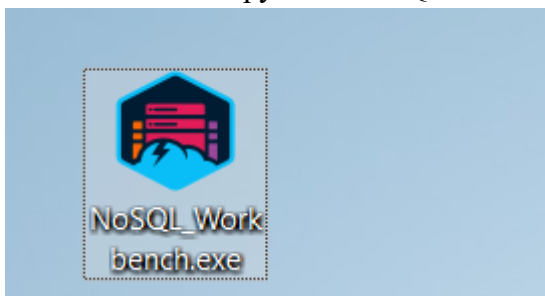
```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb create-table \
> --table-name sonkkstable2 \
> --attribute-definitions AttributeName=id,AttributeType=S \
> --key-schema AttributeName=id,KeyType=HASH \
> --provisioned-throughput ReadCapacityUnits=5,WriteCapacityUnits=5 \
> --region us-east-1
{
  "TableDescription": {
    "TableArn": "arn:aws:dynamodb:us-east-1:418862088861:table/sonkkstable2",
    "AttributeDefinitions": [
      {
        "AttributeName": "id",
        "AttributeType": "S"
      }
    ],
    "ProvisionedThroughput": {
      "NumberOfDecreasesToday": 0,
      "WriteCapacityUnits": 5,
      "ReadCapacityUnits": 5
    },
    "TableSizeBytes": 0,
    "TableName": "sonkkstable2",
    "TableStatus": "CREATING",
    "TableId": "c5dd904c-19e2-49f5-a5b4-5ac8caffd9d2",
    "KeySchema": [
      {
        "KeyType": "HASH",
        "AttributeName": "id"
      }
    ],
    "ItemCount": 0,
    "CreationDateTime": 1771774397.395
  }
}
```

Таблиця створилась

```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb list-tables
{
  "TableNames": [
    "sonkkstable",
    "sonkkstable2"
  ]
}
```

NOSQL Workbench

Встановлюю інструмент NoSQL Workbench для моделювання таблиці DynamoDB



Налаштовую модель таблиці sonkkstable3 у NoSQL Workbench та задаю атрибути майбутньої таблиці (брала атрибути з датасету з минулої лабораторної)

Table configuration

Table name *

sonkkstable3

Partition key *

The partition key is part of the table's primary key. It is a hash value that is used to retrieve items from your table and allocate data across hosts for scalability and availability. [Learn more](#)

id

String

Select sample data type (optional)

Sort key – optional

You can use a sort key as the second part of a table's primary key. The sort key allows you to sort or search among all items sharing the same partition key.

Enter sort key name

String

Select sample data type (optional)

Capacity settings

DynamoDB tables use on-demand capacity mode by default so you only pay for what you use. [Learn more about DynamoDB capacity modes](#)

☒ Use on-demand capacity mode (default)

Additional attributes - optional

Define additional base table attributes to help visualize and optimize your data model.

Attribute name *

Attribute type

Sample data type – optional ⓘ

id2 / 255

String

Select sample data format

quality7 / 255

Number

Select sample data format

alcohol7 / 255

Number

Select sample data format

Готова таблиця

sonkkstable3

Primary key configuration

Partition key name	Partition key type	Sample data type
table	String	

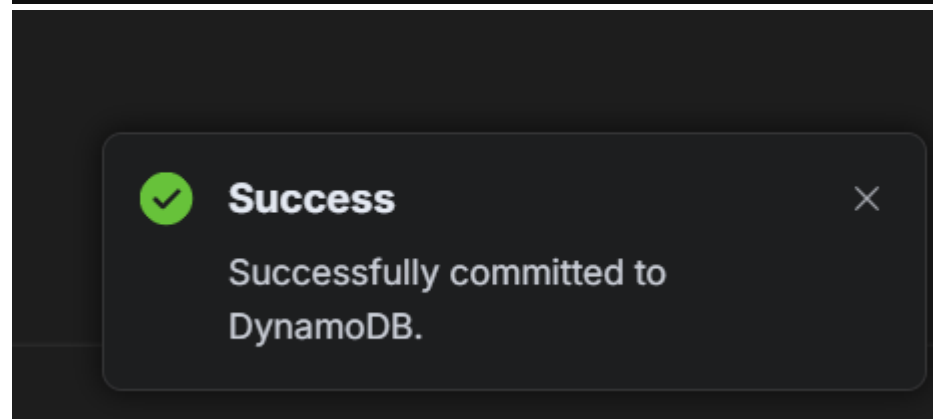
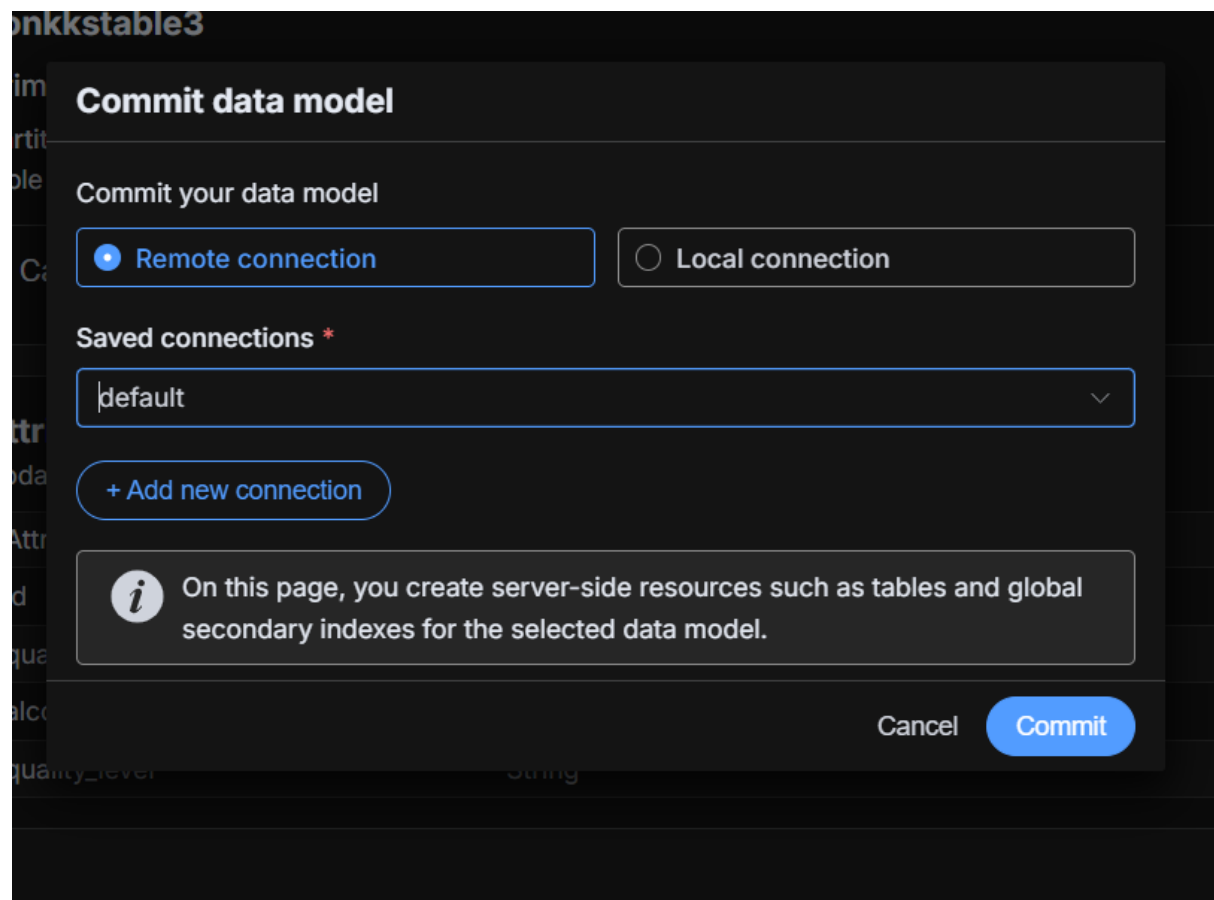
Capacity settings

Attribute definitions

Update attribute definitions for more accurate model validation.

Attribute name	Attribute type	In facets	Sample data format
id	String		
quality	Number		
alcohol	Number		
quality_level	String		

Публікую створену модель із NoSQL Workbench до Amazon DynamoDB через Commit data model



успішно додано!

```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb list-tables
{
  "TableNames": [
    "sonkkstable",
    "sonkkstable2",
    "sonkkstable3"
  ]
}
```

2. Відпрацювати додавання, видалення, модифікацію та пошук даних засобами AWS

Management Console та AWS CLI (обома способами)

AWS Management Console

ДОДАВАННЯ ДАНИХ, я додала запис до таблиці із заданими атрибутами

DynamoDB > Explore items: sonkkstable3 > Create item

Create item

FormJSON view

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attributes

Add new attribute

Attribute name	Value	Type	Remove
table - Partition key	wine1	String	
quality	7	Number	Remove
alcohol	9.5	Number	Remove
quality_level	High	String	Remove

Cancel

Create item

МОДИФІКАЦІЯ, в розділі edit item можна змінити дані атрибутів

Edit item

FormJSON view

You can add, remove, or edit the attributes of an item. You can nest attributes inside other attributes up to 32 levels deep. [Learn more](#)

Attributes

Add new attribute

Attribute name	Value	Type	Remove
table - Partition key	wine1	String	
alcohol	9.5	Number	Remove
quality	8	Number	Remove
quality_level	High	String	Remove

Cancel

Save

Save and close

ВИДАЛЕННЯ

Table: sonkkstable3 - Items returned (1/1)

Scan started on February 22, 2026, 23:16:38

Actions

Create item

Edit item

Duplicate item

Delete items

Download selected items to CSV

Download results to CSV

☒	table (String)	alcohol	quality	quality_level
☒	wine1	9.5	8	High

Table: sonkkstable3 - Items returned (0)

Query started on February 22, 2026, 23:28:10

Actions

Create item

< 1 >

table (String)

No items

No items to display.

Create item

ПОШУК, я виконую пошук даних у таблиці за допомогою Query. У результаті бачимо запис, що відповідає заданому ключу

sonkkstable3 Autopreview View

▼ Scan or query items

☐ Scan ☒ Query

Select a table or index: Table - sonkkstable3

Select attribute projection: All attributes

Partition key

Attribute: table Value: wine1

► Filters - optional

Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCLUs consumed: 0,5

Table: sonkkstable3 - Items returned (1) Actions

Query started on February 22, 2026, 23:28:10

	table (String)	alcohol	quality	quality_level
☐	wine1	9.5	8	High

AWS CLI

Додавання

```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb put-item \
--table-name sonkkstable3 \
--item '{
  "table": {"S": "wine2"},
  "quality": {"N": "5"},
  "alcohol": {"N": "8.8"},
  "quality_level": {"S": "Low"}
}' \
--region us-east-1
```

додала 2 записи в таблицю sonkkstable3 командою put-item, ось результат

Table: sonkkstable3 - Items returned (2)

Scan started on February 22, 2026, 23:40:14

	table (String)	alcohol	quality	quality_level
☐	wine1	9.5	7	High
☐	wine2	8.8	5	Low

Пошук

Виконала пошук запису за ключем через AWS CLI командою get-item

```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb get-item \
> --table-name sonkkstable3 \
> --key '{"table": {"S": "wine1"}}' \
> --region us-east-1
{
  "Item": {
    "table": {
      "S": "wine1"
    },
    "quality": {
      "N": "7"
    },
    "quality_level": {
      "S": "High"
    },
    "alcohol": {
      "N": "9.5"
    }
  }
}
```

Модифікація

```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb update-item --table-name sonkkstable3 --key '{"table": {"S": "wine1"}}' --update-expression "SET quality = :q" --expression-attribute-values '{":q":{"N":"9"}}' --region us-east-1
```

Було

☐	table (String)	alcohol	quality	quality_level
☐	wine1	9.5	7	High

Стало

☐	wine1	9.5	9	High
--------------------------	-----------------------	-----	---	------

Видалення

```
[ec2-user@ip-172-31-64-139 ~]$ aws dynamodb delete-item \
> --table-name sonkkstable3 \
> --key '{"table": {"S": "wine2"}}' \
> --region us-east-1
```

Table: sonkkstable3 - Items returned (1)

Scan started on February 22, 2026, 23:46:51

☐	table (String)	alcohol	quality	quality_level
☐	wine1	9.5	9	High

3. Для вибраних даних в JSON-форматі засобами Python SDK (приклад для boto3

доступні за посиланням <https://000060.awsstudygroup.com/3-gettingstartedwithawssdk/>)

виконати завдання:

- a. Створити таблицю
- b. Записати кілька семплів в таблицю
- c. Прочитати дані з таблиці
- d. Оновити частину даних
- e. Видалити запис за критерієм
- f. Завантажити приклади даних з json-файлу
- g. Запустити запит на пошук даних (не scan)
- h. Запустити scan усієї таблиці
- i. Видалити таблицю

Для початку я вирішила створити свій файл .json з книжками

```
books.json X
S: > навчання > cloudtech > лаб3 > books.json
1 [{"genre": "dark_academia", "id": "b01", "title": "Ninth House", "author": "Leigh Bardugo", "year": 2019, "rating": 9},
2 [{"genre": "dark_academia", "id": "b02", "title": "The Secret History", "author": "Donna Tartt", "year": 1992, "rating": 10},
3 [{"genre": "thriller", "id": "b03", "title": "Gone Girl", "author": "Gillian Flynn", "year": 2012, "rating": 8},
4 [{"genre": "romance", "id": "b04", "title": "The Hating Game", "author": "Sally Thorne", "year": 2016, "rating": 7},
5 [{"genre": "fantasy", "id": "b05", "title": "Mistborn", "author": "Brandon Sanderson", "year": 2006, "rating": 9},
6 [{"genre": "fantasy", "id": "b06", "title": "The Name of the Wind", "author": "Patrick Rothfuss", "year": 2007, "rating": 9},
7 [{"genre": "dystopia", "id": "b07", "title": "1984", "author": "George Orwell", "year": 1949, "rating": 10},
8 [{"genre": "dystopia", "id": "b08", "title": "Brave New World", "author": "Aldous Huxley", "year": 1932, "rating": 8},
9 [{"genre": "mystery", "id": "b09", "title": "The Girl with the Dragon Tattoo", "author": "Stieg Larsson", "year": 2005, "rating": 8},
10 [{"genre": "mystery", "id": "b10", "title": "Big Little Lies", "author": "Liane Moriarty", "year": 2014, "rating": 9}
11
12
```

a. Створити таблицю

Ось частина коду яка відповідає за створення таблиці, я створила таблицю BooksLab3 через boto3. Задала ключі: genre як Partition key і id як Sort key, щоб потім можна було робити нормальний query по жанру

```
# a) Створити таблицю
def create_table():
    print("a) create_table...")
    table = dynamodb.create_table(
        TableName=TABLE,
        KeySchema=[
            {"AttributeName": "genre", "KeyType": "HASH"},
            {"AttributeName": "id", "KeyType": "RANGE"}
        ],
        AttributeDefinitions=[
            {"AttributeName": "genre", "AttributeType": "S"},
            {"AttributeName": "id", "AttributeType": "S"}
        ],
        BillingMode="PAY_PER_REQUEST"
```

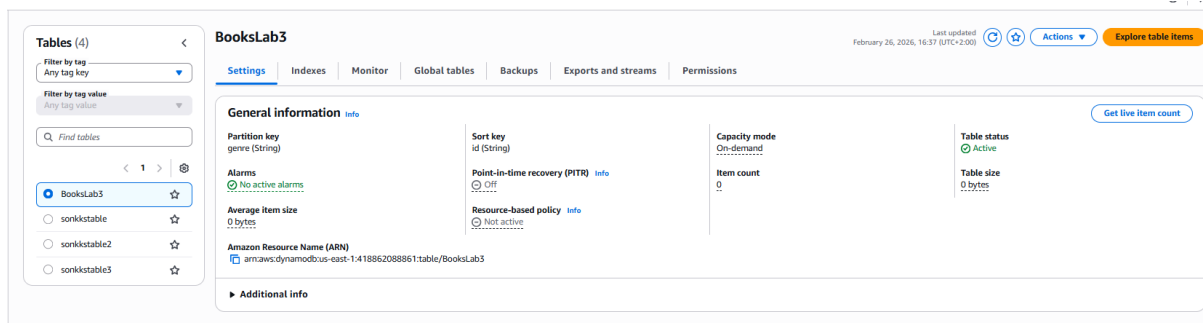
```
)
table.wait_until_exists()
print("Table created:", TABLE)
```

```
def main():
    # a
    create_table()
    table = wait_table(TABLE)
```

Вивід

```
S:\навчання\cloudtech\лаб3>py lab3.py
a) create_table...
Table created: BooksLab3
```

Бачимо, що таблиця створена



b. Записати кілька семплів в таблицю

Я додатково вручну додала 2 записи через `put_item` (це як тестові семпли), щоб показати, що я вмію вставляти елементи не тільки з файлу. (так як спершу я зробила пункт f, завантаживши дані з файлу `books.json`)

```
# b) Записати кілька семплів (додатково)
def put_samples(table):
    print("b) put_item samples...")
    samples = [
        {"genre": "fantasy", "id": "b11", "title": "The Hobbit", "author": "J.R.R. Tolkien", "year": 1937, "rating": 9},
        {"genre": "mystery", "id": "b12", "title": "The Da Vinci Code", "author": "Dan Brown", "year": 2003, "rating": 7}
    ]
    for it in samples:
        table.put_item(Item=it)
    print("Inserted:", len(samples))

# b
put_samples(table)
```

Вивід, бачимо що додвлись 2 записи

```
S:\навчання\cloudtech\лаб3>py lab3.py
a) create_table...
Table created: BooksLab3
f) load from json file: books.json
Loaded items: 10
b) put_item samples...
Inserted: 2
```

Ось бачимо 12 записів (10 з файлу + 2 додані вручну)

Table: BooksLab3 - Items returned (12)							
Scan started on February 26, 2026, 16:45:44							
	genre (String)	id (String)	author	rating	title	year	
☐	dystopia	b07	George Or...	10	1984	1949	
☐	dystopia	b08	Aldous Hux...	8	Brave New ...	1932	
☐	fantasy	b05	Brandon Sa...	10	Mistborn	2006	
☐	fantasy	b06	Patrick Rot...	9	The Name ...	2007	
☐	fantasy	b11	J.R.R. Tolkien	9	The Hobbit	1937	
☐	thriller	b03	Gillian Flynn	8	Gone Girl	2012	
☐	romance	b04	Sally Thorne	7	The Hating ...	2016	
☐	dark_academia	b01	Leigh Bard...	9	Ninth House	2019	
☐	dark_academia	b02	Donna Tartt	10	The Secret ...	1992	
☐	mystery	b09	Stieg Larsson	8	The Girl wit...	2005	
☐	mystery	b10	Liane Moria...	9	Big Little Lies	2014	
☐	mystery	b12	Dan Brown	7	The Da Vinc...	2003	

с. Прочитати дані з таблиці

Я прочитала один конкретний запис через `get_item`. Для цього вказала ключі `genre='fantasy'` і `id='b05'`, і отримала всю інформацію по цій книжці

```
# с) Прочитати дані з таблиці
def read_one(table, genre, book_id):
    print("c) get_item...")
    resp = table.get_item(Key={"genre": genre, "id": book_id})
    item = resp.get("Item")
    print("Got item:", item)
```

```
# с
read_one(table, "fantasy", "b05")
```

вивід, бачимо інформацію по книзі

```
S:\навчання\cloudtech\лаб3>py lab3.py
a) create_table...
Table created: BooksLab3
f) load from json file: books.json
Loaded items: 10
b) put_item samples...
Inserted: 2
c) get_item...
Got item: {'year': Decimal('2006'), 'genre': 'fantasy', 'rating': Decimal('9'), 'title': 'Mistborn', 'author': 'Brandon Sanderson', 'id': 'b05'}
```

Ось як виглядає цей запис в інтерфейсі Explore table items

Table: BooksLab3 - Items returned (12)

Scan started on February 26, 2026, 16:45:44

☐	genre (String)	id (String)	author	rating	title	year
☐	dystopia	b07	George Or...	10	1984	1949
☐	dystopia	b08	Aldous Huxl...	8	Brave New ...	1932
☐	fantasy	b05	Brandon Sa...	10	Mistborn	2006
☐	fantasy	b06	Patrick Rot...	9	The Name ...	2007
☐	fantasy	b11	J.R.R. Tolkien	9	The Hobbit	1937
☐	thriller	b03	Gillian Flynn	8	Gone Girl	2012

d. Оновити частину даних

Я оновила одне поле в записі через `update_item`. Для книги b05 я змінила rating на 10. Потім я ще раз зробила `get_item`, щоб перевірити, що оновлення реально збереглося

```
# d) Оновити частину даних
def update_item(table, genre, book_id):
    print("d) update_item (rating -> 10)...")
    resp = table.update_item(
        Key={"genre": genre, "id": book_id},
        UpdateExpression="SET rating = :r",
        ExpressionAttributeValues={":r": 10},
        ReturnValues="ALL_NEW"
    )
    print("Updated item:", resp["Attributes"])
```

```
# d
update_item(table, "fantasy", "b05")
```

Ось можемо побачити, що оцінка після цього пункту змінилась (було 9, стало 10)

```
c) get_item...
Got item: {'year': Decimal('2006'), 'genre': 'fantasy', 'rating': Decimal('9'), 'title': 'Mistborn', 'author': 'Brandon Sanderson', 'id': 'b05'}
d) update_item (rating -> 10)...
Updated item: {'year': Decimal('2006'), 'id': 'b05', 'genre': 'fantasy', 'rating': Decimal('10'), 'author': 'Brandon Sanderson', 'title': 'Mistborn'}
Check after update:
c) get_item...
Got item: {'author': 'Brandon Sanderson', 'genre': 'fantasy', 'rating': Decimal('10'), 'title': 'Mistborn', 'year': Decimal('2006'), 'id': 'b05'}
```

e. Видалити запис за критерієм

Я видалила один запис з таблиці методом `delete_item`. Як критерій я використала ключі запису `genre='mystery'` та `id='b12'`, тобто видалила конкретний елемент по його primary key

```
# e) Видалити запис за критерієм
def delete_one(table, genre, book_id):
    print("e) delete_item...")
    key = {"genre": genre, "id": book_id}
    table.delete_item(Key=key)
```

```
print("Deleted item:", key)
```

```
# e
delete_one(table, "mystery", "b12")
```

Вивід видалення

```
e) delete_item...
Deleted item: {'genre': 'mystery', 'id': 'b12'}
g) query (genre = fantasy )
```

Бачимо, що 12го запису немає

Table: BooksLab3 - Items returned (11)

Scan started on February 26, 2026, 17:01:15

☐	genre (String)	id (String)	author	rating	title	year
☐	dystopia	b07	George Or...	10	1984	1949
☐	dystopia	b08	Aldous Huxl...	8	Brave New ...	1932
☐	fantasy	b05	Brandon Sa...	10	Mistborn	2006
☐	fantasy	b06	Patrick Rot...	9	The Name ...	2007
☐	fantasy	b11	J.R.R. Tolkien	9	The Hobbit	1937
☐	thriller	b03	Gillian Flynn	8	Gone Girl	2012
☐	romance	b04	Sally Thorne	7	The Hating ...	2016
☐	dark_academia	b01	Leigh Bardu...	9	Ninth House	2019
☐	dark_academia	b02	Donna Tartt	10	The Secret ...	1992
☐	mystery	b09	Stieg Larsson	8	The Girl wit...	2005
☐	mystery	b10	Liane Moria...	9	Big Little Lies	2014

f. Завантажити приклади даних з json-файлу

Я зчитала дані з файлу books.json і завантажила всі елементи в таблицю через batch_writer. Так швидше додати багато записів одразу

```
# f) Завантажити приклади даних з json-файлу
def load_from_json(table, filename="books.json"):
    print("f) load from json file:", filename)
    with open(filename, "r", encoding="utf-8") as f:
        items = json.load(f)

    with table.batch_writer() as batch:
        for it in items:
            batch.put_item(Item=it)

    print("Loaded items:", len(items))
```

```
# f
load_from_json(table, "books.json")
```

Вивід

```
S:\навчання\cloudtech\лаб3>py lab3.py
a) create_table...
Table created: BooksLab3
f) load from json file: books.json
Loaded items: 10
```

Ось бачимо записи (10 з файлу + 2 додані вручну)

Table: BooksLab3 - Items returned (12)						
Scan started on February 26, 2026, 16:45:44						
☐	genre (String)	id (String)	author	rating	title	year
☐	dystopia	b07	George Or...	10	1984	1949
☐	dystopia	b08	Aldous Huxl...	8	Brave New ...	1932
☐	fantasy	b05	Brandon Sa...	10	Mistborn	2006
☐	fantasy	b06	Patrick Rot...	9	The Name ...	2007
☐	fantasy	b11	J.R.R. Tolkien	9	The Hobbit	1937
☐	thriller	b03	Gillian Flynn	8	Gone Girl	2012
☐	romance	b04	Sally Thorne	7	The Hating ...	2016
☐	dark_academia	b01	Leigh Bardu...	9	Ninth House	2019
☐	dark_academia	b02	Donna Tartt	10	The Secret ...	1992
☐	mystery	b09	Stieg Larsson	8	The Girl wit...	2005
☐	mystery	b10	Liane Moria...	9	Big Little Lies	2014
☐	mystery	b12	Dan Brown	7	The Da Vinc...	2003

g. Запустити запит на пошук даних (не scan)

Я зробила пошук через query, а не через scan. Я шукала всі записи, в яких genre = 'fantasy'. Для цього я використала KeyConditionExpression, тобто пошук йде по partition key і працює швидше, ніж scan

```
# g) Запустити query (не scan)
def query_by_genre(table, genre):
    print("g) query (genre =", genre, ")")
    resp = table.query(
        KeyConditionExpression=Key("genre").eq(genre)
    )
    items = resp.get("Items", [])
    print("Query count:", len(items))
    for it in items:
        print(" -", it["id"], it["title"], "| rating:", it.get("rating"))
```

```
# g
query_by_genre(table, "fantasy")
```

У консолі виводиться Query count і нижче список знайдених елементів з їх назвами та рейтингом

```
g) query (genre = fantasy )
Query count: 3
- b05 Mistborn | rating: 10
- b06 The Name of the Wind | rating: 9
- b11 The Hobbit | rating: 9
```

В dynamoDB виставляю пошук режим Query з параметром genre = fantasy, бачимо ті самі елементи

BooksLab3

▼ Scan or query items

☐ Scan ☒ Query

Select a table or index: Table - BooksLab3

Select attribute projection: All attributes

Partition key

Attribute: genre

Value: fantasy

Sort key - optional

Attribute: id

Value: Equal to

Enter attribute value

► Filters - optional

Run Reset

Completed - Items returned: 3 - Items scanned: 3 - Efficiency: 100% - RCUs consumed: 0.5

Table: BooksLab3 - Items returned (3)

Query started on February 26, 2026, 17:06:34

	genre (String)	id (String)	author	rating	title	year
☐	fantasy	b05	Brandon Sa...	10	Mistborn	2006
☐	fantasy	b06	Patrick Rot...	9	The Name ...	2007
☐	fantasy	b11	J.R.R. Tolkien	9	The Hobbit	1937

h. Запустити scan усієї таблиці

Я виконала scan, щоб отримати всі елементи таблиці, незалежно від значень ключів. Тобто scan просто проходиться по всій таблиці і повертає всі записи

```
# h) Scan усієї таблиці
def scan_all(table):
    print("h) scan all...")
    resp = table.scan()
    items = resp.get("Items", [])
    print("Scan count:", len(items))
```

```
# h
scan_all(table)
```

У консолі виводиться кількість елементів, які є в таблиці на момент виконання

```
h) scan all...
Scan count: 11
```

В AWS Console у вкладці Explore table items я відкрила таблицю в режимі Scan / перегляд усіх items і побачила всі записи, які є в таблиці

BooksLab3

▼ Scan or query items

☒ Scan ☐ Query

Select a table or index: Table - BooksLab3

Select attribute projection: All attributes

► Filters - optional

Completed - Items returned: 11 - Items scanned: 11 - Efficiency: 100% - RCUs consumed: 2

Table: BooksLab3 - Items returned (11)
Scan started on February 26, 2026, 17:09:17

	genre (String)	id (String)	author	rating	title	year
☐	dystopia	b07	George Or...	10	1984	1949
☐	dystopia	b08	Aldous Huxl...	8	Brave New ...	1932
☐	fantasy	b05	Brandon Sa...	10	Mistborn	2006
☐	fantasy	b06	Patrick Rot...	9	The Name ...	2007
☐	fantasy	b11	J.R.R. Tolkien	9	The Hobbit	1937
☐	thriller	b03	Gillian Flynn	8	Gone Girl	2012
☐	romance	b04	Sally Thorne	7	The Hating ...	2016
☐	dark_academia	b01	Leigh Bardu...	9	Ninth House	2019
☐	dark_academia	b02	Donna Tartt	10	The Secret ...	1992
☐	mystery	b09	Stieg Larsson	8	The Girl wit...	2005
☐	mystery	b10	Liane Moria...	9	Big Little Lies	2014

i. Видалити таблицю

Цей пункт я зробила окремим скриптом, щоб мати змогу перевірити всі попередні пункти і подивитись дані в інтерфейсі DynamoDB
я звернулась до таблиці BooksLab3 і видалила її методом delete. Потім я додала wait_until_not_exists, щоб дочекатися, поки таблиця реально зникне

```
import boto3
import botocore

REGION = "us-east-1"
TABLE = "BooksLab3"

dynamodb = boto3.resource("dynamodb", region_name=REGION)

print("i) delete_table...")

try:
```

```

table = dynamodb.Table(TABLE)
table.delete()
table.wait_until_not_exists()
print("Table deleted:", TABLE)
except botocore.exceptions.ClientError as e:
    print("AWS error:", e.response["Error"]["Message"])

```

У консолі бачимо інформацію про видалення таблиці

```

S:\навчання\cloudtech\лаб3>py delete.py
i) delete_table...
Table deleted: BooksLab3

```

Бачимо тільки попередні таблиці, наша таблиця видалилась

Tables (3) Info							
Find tables				Filter by tag Any tag key ▾			
☐	Name ▲	Status ▼	Partition key ▼	Sort key ▼	Indexes ▼	Replication Re	
☐	sonkkstable	✓ Active	table (S)	-	0	0	
☐	sonkkstable2	✓ Active	id (S)	-	0	0	
☐	sonkkstable3	✓ Active	table (S)	-	0	0	

Висновки:

Під час виконання лабораторної роботи я навчилась працювати з Amazon DynamoDB і зрозуміла, як зберігаються дані в NoSQL таблиці. Я створила таблицю, додала в неї тестові записи та завантажила дані з JSON-файлу. Також я навчилась читати дані, змінювати окремі поля та видаляти записи за заданими ключами. Для пошуку даних я використала query, а для перегляду всіх записів — scan, і побачила різницю між ними. В кінці я видалила таблицю окремим скриптом і перевірила, що вона зникла в DynamoDB.